

Sección 7: Domain-Driven Design Táctico

El dominio deja de ser un contenedor de datos y se convierte en el centro de las decisiones del sistema.

- **Entidades:** objetos con identidad que persiste en el tiempo — Account es la misma cuenta aunque cambie su saldo.
- **Value Objects:** objetos sin identidad, definidos por sus valores — Money no tiene ID, dos Money(100, ARS) son el mismo.
- **Aggregates:** fronteras de consistencia — Account, Transaction y Customer son raíces independientes. Dentro del aggregate, relaciones de objeto. Entre aggregates, referencia por ID.
- **Domain Services:** lógica que no pertenece a una sola entidad — TransferDomainService opera entre dos cuentas.
- **Application Services:** orquestación sin lógica de negocio — cargan, delegan al dominio, persisten.
- **Domain Events:** desacoplamiento dentro del dominio — TransactionExecutedEvent vía AbstractAggregateRoot, el aggregate publica, los listeners reaccionan.
- **Repository como abstracción:** el dominio define lo que necesita (DomainAccountRepository), no lo que JPA ofrece.
- **Modelo rico:** la lógica vive en la entidad — `account.withdraw(money)` en vez de `service.setBalance()`.

La Sección 6 miraba hacia los bordes y las conexiones. La Sección 7 mira hacia el corazón: el dominio toma el control del comportamiento y el framework queda afuera de las decisiones de negocio.

84. ¿Qué es DDD y por qué importa?

→ ¿Qué es?

La estructura de tu código debería reflejar cómo funciona el negocio, no cómo funciona el framework.

No es un framework, no es una librería, no es una arquitectura. Es una filosofía de diseño donde el dominio manda.

→ Dos formas de distribuir responsabilidades



MVC clásico

Lógica de negocio en el **Service**

Entity = modelo de datos

Simple, directo, rápido

```
service.setBalance(...)
```



DDD

Lógica de negocio en el **Dominio**

Entity = objeto con comportamiento

Más estructura, más testeable

```
account.withdraw(money)
```

85. Entidades: identidad que persiste

→ ¿Qué define a una Entity?

 Entity	 Value Object
Se compara por identidad (ID)	Se compara por valor
Dos cuentas con el mismo saldo, mismo tipo, mismo estado...	Dos billetes de \$1000...
¿Son la misma cuenta?	¿Son el mismo billete?
No. Tienen IDs distintos.	No importa. Valen lo mismo.
	(próxima clase)

```
Account.java
1 package com.atlas.bank.atlas.account.model;
2
3 import ...
4
20
21 @Entity @ydochoadev
22 @Table(name = "accounts")
23 @Getter
24 @Setter
25 @NoArgsConstructor
26 @AllArgsConstructor
27 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
28 public class Account {
29
30     @Id
31     @GeneratedValue(strategy = GenerationType.IDENTITY)
32     @EqualsAndHashCode.Include
33     private Long id;
34
35     @Column(name = "account_number", nullable = false, unique = true)
36     private String accountNumber;
37
38     @Column(name = "owner_name", nullable = false)
39     private String ownerName;
40
41     @Column(nullable = false)
42     private String email;
```

```
Transaction.java
1 package com.atlas.bank.atlas.transaction.model;
2
3 import ...
4
28
29 @Entity @ydochoadev *
30 @Table(name = "transactions")
31 @Getter
32 @Setter
33 @NoArgsConstructor
34 @AllArgsConstructor
35 @Builder
36 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
37 public class Transaction {
38
39     @Id
40     @GeneratedValue(strategy = GenerationType.IDENTITY)
41     @EqualsAndHashCode.Include
42     private Long id;
43
44     @Column(nullable = false, length = 20)
45     @Enumerated(EnumType.STRING)
46     private TransactionType type; // DEPOSIT, WITHDRAWAL, TRANSFER
47
48     @Column(name = "source_account_id")
49     private Long sourceAccountId;
50
51     @Column(name = "target_account_id")
```

86. Value Objects: igualdad por valor - pt1 y pt2

→ NO tiene identidad propia, lo que importa es su valor.

→ ¿Qué define a un Value Object?

- Se compara por valor**
Dos objetos con los mismos atributos son iguales.
`Money(1000, ARS) == Money(1000, ARS)`
- Inmutable**
Una vez creado, no cambia. Si necesitas otro valor, creás uno nuevo.
`money.add(other)` → devuelve un Money nuevo
- Sin identidad**
No tiene ID. Existe como parte de una Entity, no por sí solo.

Se necesita un objeto de encapsule monto + moneda, inmutable y que tenga operaciones con nombre del dominio (add, sustract...)

→ En chared, crear paquete para las monedas:

```
Money.java Currency.java x
1 package com.atlas.bank.atlas.shared.model;
2
3 public enum Currency { no usages
4     ARS, no usages
5     USD no usages
6 }
```

** @Embeddable → Spring le dice a JPA que no tiene tabla propia. Si la clase tiene campos con el decorador @Column, significa que esos campos se guardan como columnas dentro de la tabla de quien lo contenga

```
Money.java x Currency.java
1 package com.atlas.bank.atlas.shared.model;
2
3 > import ...
13
14 @Embeddable 12 usages
15 @Getter
16 @NoArgsConstructor
17 @EqualsAndHashCode // Comparar por valor
18 public class Money {
19
20     @Column(nullable = false)
21     private BigDecimal amount;
22
23     @Enumerated(EnumType.STRING)
24     @Column(nullable = false, length = 3)
25     private Currency currency;
26
27     // Nadie fuera de la clase pueda realizar un new (usar factory method)
28     private Money(BigDecimal amount, Currency currency) { 2 usages
29         if (amount == null) {
30             throw new IllegalArgumentException("El monto no puede ser nulo");
31         }
32         this.amount = amount.setScale(newScale: 2, RoundingMode.HALF_UP); // redondeo hacia arriba
33         this.currency = currency;
34     }
35
36     @ public static Money of(BigDecimal amount, Currency currency) { 2 usages
37         return new Money(amount, currency);
38     }
39
40     @ public static Money zero(Currency currency) { no usages
41         return new Money(BigDecimal.ZERO, currency);
42     }
43
44     public Money add(Money other) {
45         validateSameCurrency(other);
46         return Money.of(this.amount.add(other.amount), this.currency);
47     }
```

```
49     public Money subtract(Money other) { no usages
50         validateSameCurrency(other);
51         return Money.of(this.amount.subtract(other.amount), this.currency);
52     }
53
54     public boolean isGreaterThan(Money other) { no usages
55         validateSameCurrency(other);
56         return this.amount.compareTo(other.amount) > 0;
57     }
58
59     public boolean isNegative() { no usages
60         return this.amount.compareTo(BigDecimal.ZERO) < 0;
61     }
62
63     @ private void validateSameCurrency(Money money) { 3 usages
64         if (this.currency != money.currency) {
65             throw new IllegalArgumentException("No se pueden operar montos en monedas distintas: " + this.currency + " VS " + money.currency);
66         }
67     }
68
69     @Override
70     public String toString() {
71         return "Money{" +
72             "amount=" + amount +
73             '}' +
74     }
75 }
```

88. Value Objects en el proyecto - pt1

➔ Integrar al proyecto la clase Money.

```
Account.java Money.java
32 public class Account {
42     @Column(name = "owner_name", nullable = false)
43     private String ownerName;
44
45     @Column(nullable = false)
46     private String email;
47
48     @Enumerated(EnumType.STRING)
49     @Column(nullable = false, length = 20)
50     private AccountType type; // SAVING, CHECKING
51
52     @Embedded
53     @AttributeOverrides({
54         {
55             @AttributeOverride(name = "amount", column = @Column(name = "balance", nullable = false)),
56             @AttributeOverride(name = "currency", column = @Column(name = "currency", nullable = false, length = 3))
57         }
58     })
59     private Money balance;
60
61     @Enumerated(EnumType.STRING)
```

Spring le dice a JPA que no es una columna simple (es un objeto que se guarda como columnas en la tabla => 2 datos, 2 columnas)

Nombre que viene de la clase Money

Nombre que tendrá en la tabla

```
private LocalDateTime createdAt;
8
9 @PrePersist @Transactional
10 public void prePersist() {
11     this.createdAt = LocalDateTime.now();
12     if (this.status == null) this.status = AccountStatus.ACTIVE;
13     if (this.balance == null) this.balance = Money.zero(Currency.ARS);
14 }
15 }
```

➔ En el mapper:

```
Account.java AccountMapper.java Money.java
1 package com.atlas.bank.atlas.account.dto;
2
3 import com.atlas.bank.atlas.account.model.Account;
4 import com.atlas.bank.atlas.shared.model.Currency;
5 import com.atlas.bank.atlas.shared.model.Money;
6 import org.mapstruct.Mapper;
7 import org.mapstruct.Mapping;
8 import org.mapstruct.Named;
9
10 import java.math.BigDecimal;
11
12 @Mapper(componentModel = "spring")
13 public interface AccountMapper {
14
15     @Mapping(target = "id", ignore = true)
16     @Mapping(target = "status", ignore = true)
17     @Mapping(target = "createdAt", ignore = true)
18     @Mapping(target = "balance", source = "balance", qualifiedByName = "toMoney")
19     Account toEntity(CreateAccountRequest request);
20
21     @Mapping(target = "balance", source = "balance", qualifiedByName = "toAmount")
22     AccountResponse toResponse(Account account);
23
24     @Named("toMoney")
25     default Money toMoney(BigDecimal amount) {
26         if (amount == null) return null;
27         return Money.of(amount, Currency.ARS);
28     }
29
30     @Named("toAmount")
31     default BigDecimal toAmount(Money money) {
32         if (money == null) return null;
33         return money.getAmount();
34     }
35 }
```

89. Value Objects en el proyecto - pt2

→Corrigiendo Facade:

```
AccountDashboardFacade.java Money.java
1 package com.atlas.bank.atlas.account.service;
2
3 > import ...
12
13 @Service 2 usages ydochoadev *
14 @RequiredArgsConstructor
15 public class AccountDashboardFacade {
16     private final IAccountService accountService;
17     private final ITransactionQueryService transactionQueryService;
18     private final TransactionMapper transactionMapper;
19
20     public DashboardResponse getDashboard(Long accountId) { 1 usage ydochoadev *
21         Account account = this.accountService.findById(accountId);
22         // Lista de trx
23         List<TransactionResponse> transactions = transactionQueryService
24             .getByAccountId(accountId) List<Transaction>
25             .stream() Stream<Transaction>
26             .map(transactionMapper::toResponse) Stream<TransactionResponse>
27             .toList();
28
29         return DashboardResponse.builder()
30             .accountId(account.getId())
31             .accountNumber(account.getAccountNumber())
32             .ownerName(account.getOwnerName())
33             .type(account.getType().name())
34             .balance(account.getBalance().getAmount())
35             .status(account.getStatus().name())
36             .recentTransactions(transactions)
37             .build();
38     }
39 }
```

→En TransferService:

```
AccountDashboardFacade.java TransferService.java SufficentFundsValidator.java Money.java
21 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
69     // Validar que la cuenta esté activa. Fondos. Y fraude
70     validators.forEach( TransferValidator validator -> validator.validate(context));
71 }
72
73 @Override 1 usage ydochoadev
74 protected BigDecimal calculateFee(TransferContext context) {
75     // Calcular comisión
76     return this.feeCalculators.stream() Stream<FeeCalculator>
77         .filter( FeeCalculator fc -> fc.supports(context.from().getType()))
78         .findFirst() Optional<FeeCalculator>
79         .orElseThrow(() -> new RuntimeException("No hay calculador para el tipo " + context.from().getType()))
80         .calculate(context.amount());
81 }
82
83 @Override 1 usage ydochoadev *
84 protected void execute(TransferContext context, BigDecimal fee) {
85     // Actualizar saldos
86     BigDecimal newFromBalance = context.from().getBalance().getAmount().subtract(context.amount()).subtract(fee);
87     context.from().setBalance(Money.of(newFromBalance, context.from().getBalance().getCurrency()));
88     BigDecimal newToBalance = context.to().getBalance().getAmount().add(context.amount());
89     context.to().setBalance(Money.of(newToBalance, context.to().getBalance().getCurrency()));
90     accountRepository.save(context.from());
91     accountRepository.save(context.to());
92 }
93
94 @Override 1 usage ydochoadev
95 protected Transaction save(TransferContext context, BigDecimal fee) {
96     // Crear transacción
97     Transaction transaction = TransactionFactory.createTransfer(context, fee);
98
99     return transactionRepository.save(transaction);
100 }
101 }
```

➔En SufficientFundsValidator

```
AccountDashboardFacade.java | TransferService.java | SufficientFundsValidator.java x | Money.java
1 package com.atlas.bank.atlas.transaction.validation.chain;
2
3 > import ...
7
8 @Component @ydochoadev *
9 @Order(2)
10 public class SufficientFundsValidator implements TransferValidator {
11     @Override @usage @ydochoadev *
12     public void validate(TransferContext context) {
13         if (context.from().getBalance().getAmount().compareTo(context.amount()) < 0) {
14             throw new InsufficientFundsException(
15                 context.from().getId(),
16                 context.from().getBalance().getAmount(),
17                 context.amount());
18         }
19     }
20 }
```

➔Compilar y crear cuenta. Saldrá en la BD:

Select * From accounts

Select * From accounts;

BALANCE	CREATED_AT	ID	ACCOUNT_NUMBER	EMAIL	OWNER_NAME	CURRENCY	STATUS	TYPE
10000000.00	2026-04-29 09:50:23.334299	1	ATL-3001	carlos@mail.com	Carlos	ARS	ACTIVE	SAVINGS

(1 row, 7 ms)

90. Aggregates: qué son y por qué existen

➔ ¿Cómo agrupamos estos objetos? ¿Quién es el responsable de mantener las reglas de negocio consistentes? Y la respuesta en DDD es el aggregate.

➔ ¿Qué es un Aggregate?



➔Las reglas de un Aggregate

Toda operación pasa por la raíz

Para modificar el balance, llamás a un método de Account.
No hacés `setBalance()` desde afuera.

Consistencia dentro del Aggregate

Todo lo que está adentro se actualiza junto, en la misma operación.
Balance + estado + reglas = siempre consistente.

Referencia entre Aggregates por ID

Transaction tiene `Long sourceAccountId`, no `Account sourceAccount`.
Cada Aggregate se carga y persiste de forma independiente.

➔NO existe un JPA ManyToOne en la clase Transaction para hacer referencia a Account, si no que la referencia está por ID (son independientes): Account no sabe que Transaction existe y este solo tiene el ID de la cuenta, NO la cuenta entera.

91. Aggregates en el proyecto

➔Creando cliente:

```
Customer.java  CustomerStatus.java x
1 package com.atlas.bank.atlas.customer.model;
2
3 public enum CustomerStatus { no usages
    Rename usages
4     ACTIVE, SUSPENDED, CLOSED no usages
5 }
```

```
Customer.java x  Account.java  CustomerStatus.java
1 package com.atlas.bank.atlas.customer.model;
2
3 > import ...
19
20 @Entity
21 @Table(name = "constomers")
22 @Getter
23 @Setter
24 @NoArgsConstructor
25 @AllArgsConstructor
26 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
27 public class Customer {
28
29     @Id
30     @GeneratedValue(strategy = GenerationType.IDENTITY)
31     @EqualsAndHashCode.Include
32     private Long id;
33
34     @Column(nullable = false)
35     private String name;
36
37     @Column(nullable = false, unique = true)
38     private String email;
39
40     @Column(nullable = false, length = 20)
41     @Enumerated(EnumType.STRING)
42     private CustomerStatus status;
43
44     @Column(name = "created_at", nullable = false, updatable = false)
45     private LocalDateTime createdAt;
46
47     @PrePersist
48     public void prePersist() {
49         this.createdAt = LocalDateTime.now();
50     }
51
52     public boolean isActive() { no usages
53         return this.status == CustomerStatus.ACTIVE;
54     }
55 }
```

**** Agregar en Money:**

```
Account.java Money.java x
18 public class Money {
48
49     public Money subtract(Money other) { 1 usage 2 ydochoadev
50         validateSameCurrency(other);
51         return Money.of(this.amount.subtract(other.amount), this.currency);
52     }
53
54     public boolean isGreaterThan(Money other) { no usages 2 ydochoadev
55         validateSameCurrency(other);
56         return this.amount.compareTo(other.amount) > 0;
57     }
58
59     public boolean isLessThan(Money other) { no usages new *
60         validateSameCurrency(other);
61         return this.amount.compareTo(other.amount) < 0;
62     }
63
64     public boolean isNegative() { 2 usages 2 ydochoadev
65         return this.amount.compareTo(BigDecimal.ZERO) < 0;
66     }
67
68 @ private void validateSameCurrency(Money money) { 4 usages 2 ydochoadev
```

→ Colocar en Account:

```
Project v + - Account.java x Money.java
v atlas D:\new_projects\java\atlas
> .idea
> .mvn
> docker
v src
  v main
    v java
      v com.atlas.bank.atlas
        v account
          > controller
          > dto
          > exception
          v model
            Account
            AccountStatus
            AccountType
            repository
            service
          v customer.model
            Customer
            CustomerStatus
          v shared
            config
            exception
            model
              Currency
              Money
            transaction
            AtlasApplication
          resources
          test
          target
.gitattributes
.gitignore

34 public class Account {
65 @ private AccountStatus status; // ACTIVE, CLOSED, FROZEN
66
67 @Column(name = "created_at", nullable = false, updatable = false)
68 @ private LocalDateTime createdAt;
69
70 @Column(name = "customer_id")
71 @ private Long customerId;
72
73 @PrePersist 2 ydochoadev
74 public void prePersist() {
75     this.createdAt = LocalDateTime.now();
76     if (this.status == null) this.status = AccountStatus.ACTIVE;
77     if (this.balance == null) this.balance = Money.zero(Currency.ARS);
78 }
79
80 @ public void deposit(Money money) { no usages new *
81     if (money.isNegative()) {
82         throw new IllegalArgumentException("El monto a depositar no puede ser negativo");
83     }
84     this.balance = this.balance.add(money);
85 }
86
87 @ public void withdraw(Money money) { no usages new *
88     if (money.isNegative()) {
89         throw new IllegalArgumentException("El monto a retirar no puede ser negativo");
90     }
91     if (this.balance.isLessThan(money)) {
92         throw new InsufficientFundsException(id, this.balance.getAmount(), money.getAmount());
93     }
94
95     this.balance = this.balance.subtract(money);
96 }
97 }
```

92. Aggregates: relaciones entre raíces

Definimos tres aggregates. Cada uno tiene su identidad y su ciclo de vida.

Pero no funcionan aislados. ¿Cómo se conectan entre sí?



Una transacción necesita saber de qué cuenta sale. Una cuenta necesita saber a qué cliente pertenece.

El camino tentador: @ManyToOne

```
// ❌ Transaction con @ManyToOne a Account
```

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "source_account_id")
private Account sourceAccount;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "target_account_id")
private Account targetAccount;
```

```
// ❌ Account con @ManyToOne a Customer
```

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "customer_id")
private Customer customer;
```

- 1 JPA **arrastra al otro aggregate** a la misma sesión de Hibernate. Dos raíces independientes quedan acopladas en una sola unidad de trabajo.
- 2 Si modificas Customer mientras operas con Account, Hibernate puede **persistir ese cambio automáticamente** por dirty checking — sin que se lo pidas.
- 3 La **frontera de consistencia desaparece**. Cada aggregate debería ser una transacción independiente. Con @ManyToOne, se convierten en un solo bloque.

→ ¿Qué es dirty checking?

Cuando Hibernate carga una entidad, guarda una "foto" de su estado original.

Al hacer flush o commit, compara el estado actual contra esa foto. Si algo cambió, **genera un UPDATE automáticamente**.



¿Lazy o eager? El problema es el mismo. Con lazy, solo se posterga al momento en que accedés a la relación.

→ La regla: Dentro VS Entre aggregates

"Dentro del aggregate, **relaciones de objeto**.
Fuera del aggregate, **relación por ID**."

DENTRO del aggregate

Relaciones de objeto



```
// ✅ OrderItem NO es aggregate root
@OneToMany(cascade = ALL)
private List<OrderItem> items;
```

Se cargan juntos, se persisten juntos

ENTRE aggregates

Referencia por ID



```
// ✅ Customer ES aggregate root
@Column(name = "customer_id")
private Long customerId;
```

Se cargan por separado, cada uno por su repositorio

atlas-bank ya lo hace bien

Transaction.java

```
// ✅ Referencia por ID, no por objeto

@Column(name = "source_account_id")
private Long sourceAccountId;

@Column(name = "target_account_id")
private Long targetAccountId;
```

Transaction no carga las cuentas.
Solo conoce sus IDs.

Account.java

```
// ✅ Referencia por ID, no por objeto

@Column(name = "customer_id")
private Long customerId;
```

Account no arrastra al Customer.
Son dos aggregates independientes.

¿Y si necesito datos del Customer?

El service lo busca aparte:

```
customerRepository.findById(account.getCustomerId())
```

No es ~~Account sourceAccount~~. Es **Long sourceAccountId**. Eso es referencia por ID.

93. Domain services: Lógica entre entidades

→ En una trx, se debe coordinar 2 cuentas, retirar en una y depositar en otra. Esa lógica no pertenece a ninguna de las dos cuentas. Se coloca esa responsabilidad en un Domain Service.

→ ¿Cuándo usar un Domain Service?

Involucra más de una entity

Una transferencia toca dos cuentas.

Ninguna de las dos es "dueña" de la operación completa.

No pertenece a ninguna entity

¿`from.transferTo(to, amount)`? No — Account no debería conocer a otra Account.

Es lógica de negocio, no orquestación

Débito + crédito = negocio.

Cargar datos + persistir + publicar eventos = orquestación.

TransactionDomainService.java

```
1 package com.atlas.bank.atlas.transaction.service.domain;
2
3 import com.atlas.bank.atlas.account.model.Account;
4 import com.atlas.bank.atlas.shared.model.Money;
5 import org.springframework.stereotype.Service;
6
7 import java.math.BigDecimal;
8
9 @Service
10 public class TransactionDomainService {
11
12     @
13     public void transfer(Account fromAccount, Account toAccount, BigDecimal amount, BigDecimal fee) {
14         Money totalDebit = Money.of(amount.add(fee), fromAccount.getBalance().getCurrency());
15         Money depositAmount = Money.of(amount, toAccount.getBalance().getCurrency());
16         fromAccount.withdraw(totalDebit);
17         toAccount.deposit(depositAmount);
18     }
19 }
```

→ En el servicio de la trx.

```
1 package com.atlas.bank.atlas.transaction.service.transfer;
2
3 > import ...
19
20 @Service @ydochoadev *
21 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
22
23     private final AccountRepository accountRepository; 5 usages
24     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones 2 usages
25     private final ApplicationEventPublisher eventPublisher; 2 usages
26     private final List<TransferValidator> validators; 2 usages
27     private final TransferDomainService transferDomainService; 2 usages
28
29     public TransferService(TransactionRepository transactionRepository, @ydochoadev *
30         AccountRepository accountRepository,
31         List<FeeCalculator> feeCalculators,
32         ApplicationEventPublisher eventPublisher,
33         List<TransferValidator> validators,
34         TransferDomainService transferDomainService) {
35         super(transactionRepository);
36         this.accountRepository = accountRepository;
37         this.feeCalculators = feeCalculators;
38         this.eventPublisher = eventPublisher;
39         this.validators = validators;
40         this.transferDomainService = transferDomainService;
```

```
}
@
@Override 1 usage @ydochoadev *
protected void execute(TransferContext context, BigDecimal fee) {
    transferDomainService.transfer(context.from(), context.to(), context.amount(), fee);
    accountRepository.save(context.from());
    accountRepository.save(context.to());
}
```

94. Application Services: Orquestación

→ ¿Cuál es la diferencia?

 Domain Service	 Application Service
Lógica de negocio pura	Orquesta el flujo
Sin estado	Carga datos del repositorio
No persiste	Llama al dominio
No publica eventos	Persiste resultados
No accede a infraestructura	Publica eventos
<code>from.withdraw(amount)</code> <code>to.deposit(amount)</code>	<i>El director de orquesta: no toca instrumentos, los coordina</i>

** infraestructura, cuando realiza una operación en la BD.

95. Domain Events y AbstractAggregateRoot

→ ¿Quién sabe que algo pasó?

 Hoy: el Service publica El Application Service llama a <code>eventPublisher.publishEvent()</code> después de ejecutar. Riesgo: si alguien se olvida de publicar, el evento se pierde.	 DDD: el Aggregate registra La entity llama a <code>registerEvent()</code> cuando la operación ocurre. Spring lo publica automáticamente al hacer <code>save()</code> .
--	---

→ Para solucionar ello, se extiende de una clase propia de spring en la Transaction:

Cuando una Entity tiene “AbstractAggregateRoot”, puede registrar eventos con “RegisterEvent”: cuando se realiza un “save”, spring publica automáticamente todos los eventos registrados.

** `callSuper = false` => Le dice al “EqualsAndHashCode” que no intente incluir campos de la clase padre en el equals.

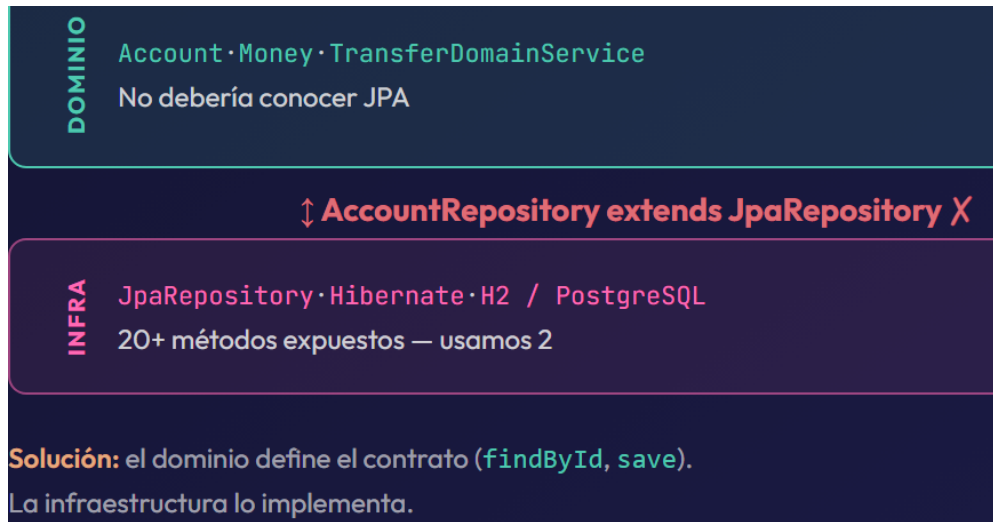
```
Transaction.java x
1  package com.atlas.bank.atlas.transaction.model;
2
3  > import ...
30
31  @Entity @ydochoadev *
32  @Table(name = "transactions")
33  @Getter
34  @Setter
35  @NoArgsConstructor
36  @AllArgsConstructor
37  @Builder
38  @EqualsAndHashCode(onlyExplicitlyIncluded = true, callSuper = false)
39  public class Transaction extends AbstractAggregateRoot<Transaction> implements Serializable {
40
41      @Id
42      @GeneratedValue(strategy = GenerationType.IDENTITY)
43      @EqualsAndHashCode.Include
44      private Long id;
45
```

En TransferService:

```
Transaction.java x TransferService.java x
19  public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
36
37
38      @Override @ydochoadev *
39      @Transactional
40      public Transaction execute(Long fromId, Long toId, BigDecimal amount) {
41          // Buscar cuentas
42          Account from = accountRepository.findById(fromId)
43              .orElseThrow(() -> new AccountNotFoundException(fromId));
44          Account to = accountRepository.findById(toId)
45              .orElseThrow(() -> new AccountNotFoundException(toId));
46
47          // process => aplica el patrón Template Method
48          Transaction transaction = process(new TransferContext(from, to, amount));
49          // Estados
50          transaction.advanceTo(transaction.getState().validate());
51          transaction.advanceTo(transaction.getState().execute());
52          // Registrar evento
53          transaction.markAsExecuted();
54          transactionRepository.save(transaction); // Guarda (actualiza) otra vez la trx con su estado. Publica evento
55
56          return transaction;
57      }
58
```

96. Repository como abstracción del dominio

→ ¿De quién es el repositorio?



Cuando la interface extiende de JpaRepository, se tienen varios métodos que no se utilizan.

→ Creando una nueva interface que tengan los métodos que se usarán (con el mismo nombre que los métodos de JPA)

```
DomainAccountRepository.java x
1 package com.atlas.bank.atlas.account.repoditory;
2
3 import com.atlas.bank.atlas.account.model.Account;
4
5 import java.util.List;
6 import java.util.Optional;
7
8 public interface DomainAccountRepository { no usages
9
10     Optional<Account> findById(Long id); no usages
11
12     List<Account> findAll(); no usages
13
14     Account save(Account account); no usages
15 }
```

→ En el repositorio original, extender, además de Jpa, del Domain creaco:

```
DomainAccountRepository.java AccountRepository.java x
1 package com.atlas.bank.atlas.account.repoditory;
2
3 > import ...
4
5
6 public interface AccountRepository extends JpaRepository<Account, Long>, DomainAccountRepository {
7
8 }
```

Como los métodos de DomainAccountRepository son los mismo de Jpa (que ya están implementados) no es necesario implementarlos.

→ En el service:

```
DomainAccountRepository.java AccountRepository.java TransferService.java AccountService.java

1 package com.atlas.bank.atlas.transaction.service.transfer;
2
3 > import ...
17
18 @Service @ydochoadev *
19 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
20
21     private final DomainAccountRepository accountRepository; // 2 usages
22     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones 2 usages
23     private final List<TransferValidator> validators; 2 usages
24     private final TransferDomainService transferDomainService; 2 usages
25
26     public TransferService(TransactionRepository transactionRepository, @ydochoadev *
27         DomainAccountRepository accountRepository,
28         List<FeeCalculator> feeCalculators,
29         List<TransferValidator> validators,
30         TransferDomainService transferDomainService) {
31         super(transactionRepository);
32         this.accountRepository = accountRepository;
33         this.feeCalculators = feeCalculators;
34         this.validators = validators;
35         this.transferDomainService = transferDomainService;
36     }
```

```
DomainAccountRepository.java AccountRepository.java TransferService.java AccountService.java

1 package com.atlas.bank.atlas.transaction.service.transfer;
2
3 > import ...
17
18 @Service @ydochoadev *
19 public class TransferService extends TransactionProcessor<TransferContext> implements ITransferService {
20
21     private final DomainAccountRepository accountRepository; // 2 usages
22     private final List<FeeCalculator> feeCalculators; // Se tiene TODA las implementaciones 2 usages
23     private final List<TransferValidator> validators; 2 usages
24     private final TransferDomainService transferDomainService; 2 usages
25
26     public TransferService(TransactionRepository transactionRepository, @ydochoadev *
27         DomainAccountRepository accountRepository,
28         List<FeeCalculator> feeCalculators,
29         List<TransferValidator> validators,
30         TransferDomainService transferDomainService) {
31         super(transactionRepository);
32         this.accountRepository = accountRepository;
33         this.feeCalculators = feeCalculators;
34         this.validators = validators;
35         this.transferDomainService = transferDomainService;
36     }
```

97. Modelo rico en el proyecto

→ Dar a Transaction manejar su propio ciclo de vida.

→ Inicio de la sección: el service hacía todo

```
// El service manipulaba balances directamente
ctx.from().setBalance(ctx.from().getBalance().subtract(amount).subtract(fee));
ctx.to().setBalance(ctx.to().getBalance().add(amount));

// El service manejaba el ciclo de vida
transaction.advanceTo(transaction.getState().validate());
transaction.advanceTo(transaction.getState().execute());

// El service publicaba eventos manualmente
eventPublisher.publishEvent(new TransactionExecutedEvent(...));
```

La entity era un contenedor de datos. El service tenía toda la lógica.

➔Final de la sección: el service orquesta

```
// EL Domain Service coordina - Account.withdraw() y Account.deposit()
transferDomainService.transfer(from, to, amount, fee);

// La Transaction maneja su ciclo de vida y registra el evento
transaction.executeTransfer();
transactionRepository.save(transaction);
```

La lógica vive en el dominio. El service solo **carga, delega y persiste**.

➔Modificar el servicio:

98. Tarea: Email como Value Object

<https://gist.github.com/gchaldou/a9dc03e26af53aec7a19d9b67c7670db>

```
1 package com.atlas.bank.atlas.shared.model;
2
3 > import ...
4
5 @Embeddable 2 usages
6 @Getter
7 @NoArgsConstructor
8 @EqualsAndHashCode
9 public class Email {
10
11     @Column(nullable = false)
12     private String value;
13
14     private Email(String value) { 1 usage
15         if (value == null || value.isBlank()) {
16             throw new IllegalArgumentException("El correo no puede ser nulo o vacio");
17         }
18
19         if (value.matches(regex: "^[\\w.-]+@[\\w.-]+\\.([a-zA-Z]{2,})$")) {
20             throw new IllegalArgumentException("El correo no tiene el formato correcto: " + value);
21         }
22         this.value = value.toLowerCase().trim();
23     }
24
25     public static Email of(String value) { no usages
26         return new Email(value);
27     }
28
29     @Override
30     public String toString() {
31         return this.value;
32     }
33 }
```

→Aplicar Email en Account y Customer

```
35 public class Account {
36
37     @EqualsAndHashCode.Include
38     private Long id;
39
40     @Column(name = "account_number", nullable = false, unique = true)
41     private String accountNumber;
42
43     @Column(name = "owner_name", nullable = false)
44     private String ownerName;
45
46     @Embedded
47     @AttributeOverrides(
48         {
49             @AttributeOverride(name = "value", column = @Column(name = "email", nullable = false))
50         }
51     )
52     private Email email;
53
54     @Enumerated(EnumType.STRING)
55     @Column(nullable = false, length = 20)
56     private AccountType type; // SAVING, CHECKING
57 }
```

```
29 @AllArgsConstructor
30 @EqualsAndHashCode(onlyExplicitlyIncluded = true)
31 public class Customer {
32
33     @Id
34     @GeneratedValue(strategy = GenerationType.IDENTITY)
35     @EqualsAndHashCode.Include
36     private Long id;
37
38     @Column(nullable = false)
39     private String name;
40
41     @Embedded
42     @AttributeOverrides(
43         {
44             @AttributeOverride(name = "value", column = @Column(name = "email", nullable = false, unique = true))
45         }
46     )
47     private Email email;
```

→ En el mapper:

```
AccountMapper.java x CreateAccountRequest.java Account.java
1 package com.atlas.bank.atlas.account.dto;
2
3 > import ...
12
13 @Mapper(componentModel = "spring") 3 usages 1 implementation ydochoadev *
14 public interface AccountMapper {
15
16     @Mapping(target = "id", ignore = true) 1 usage 1 implementation ydochoadev
17     @Mapping(target = "status", ignore = true)
18     @Mapping(target = "createdAt", ignore = true)
19     @Mapping(target = "balance", source = "balance", qualifiedByName = "toMoney")
20     @Mapping(target = "email", source = "email", qualifiedByName = "toEmail")
21     Account toEntity(CreateAccountRequest request);
22
23     @Mapping(target = "balance", source = "balance", qualifiedByName = "toAmount")
24     @Mapping(target = "email", source = "email", qualifiedByName = "fromEmail")
25     AccountResponse toResponse(Account account);
26
27     @Named("toMoney") no usages ydochoadev
28     default Money toMoney(BigDecimal amount) {
29         if (amount == null) return null;
30         return Money.of(amount, Currency.ARS);
31     }
32
33     @Named("toAmount") no usages ydochoadev
34     default BigDecimal toAmount(Money money) {
35         if (money == null) return null;
36         return money.getAmount();
37     }
38
39     @Named("toEmail") no usages new *
40     default Email toEmail(String email) {
41         if (email == null) return null;
42         return Email.of(email);
43     }
44
45     @Named("fromEmail") no usages new *
46     default String fromEmail(Email email) {
47         if (email == null) return null;
48         return email.getValue();
49     }
50 }
```